

Formulario Definitivo: Ricerca Operativa (PL, PLI, Reti)

Regole pratiche, trucchi sui segni e oltre 30 esercizi risolti

Vincenzo Pio Florio — MAT 592947 — Politecnico di Bari

Luglio 2026

Indice

1	Formulario Definitivo: Ricerca Operativa (PL, PLI, Reti)	3
1.1	Indice	3
2	PARTE A — Programmazione Lineare (Simplex in forma standard)	3
2.1	1. Forma standard e dizionario base	3
2.2	2. SBA (Soluzione di Base Ammissibile) e vincoli	4
2.3	3. La forma canonica e l'algebra matriciale	4
2.4	4. Test di ottimalità e regola dei segni	6
2.5	5. Analisi di sensitività e formule matriciali abbreviate	6
2.5.1	5.1 Sensitività sui costi (c^{new} o Δc)	6
2.5.2	5.2 Sensitività sui termini noti (Δb e b^{new})	7
2.6	6. Il duale e la regola del “naturale”	8
2.7	7. Scarti complementari (La Regola dell'Incrocio)	8
2.8	8. Branch and Bound (per PL intera)	8
2.9	9. Matrici TUM (Totalmente Unimodulari)	9
3	PARTE B — Reti	9
3.1	10. Il problema del commesso viaggiatore (TSP) e Nearest Neighbor	9
3.1.1	10.1 Spiegazione teorica (Formulazione matematica)	10
3.1.2	10.2 Risoluzione pratica: Euristiche Nearest Neighbor (Vicino più prossimo)	10
3.2	11. Il problema del cammino minimo e l'algoritmo di Dijkstra	11
3.2.1	11.1 Formulazione matematica (teoria)	11
3.2.2	11.2 Risoluzione pratica: Algoritmo di Dijkstra (pesi positivi)	12
3.3	12. Algoritmi su reti: Bellman-Ford (Cammino minimo con pesi negativi)	12
4	PARTE C — Modelli avanzati e linearizzazioni	13
4.1	13. Linearizzazione e condizioni logiche (PNL $\rightarrow$ PL)	13
4.1.1	13.1 Vincoli logici tra variabili binarie	13
4.1.2	13.2 Linearizzazione funzione obiettivo Min-Max (o Max-Min)	14
4.1.3	13.3 Linearizzazione del valore assoluto	14
4.2	14. Formulazione funzione fitness per PSO e algoritmi euristici (metodo delle penalità)	15
5	PARTE D — L'algoritmo infallibile	16
5.1	15. L'algoritmo infallibile: Risoluzione step-by-step dell'esercizio di PL	16
5.1.1	FASE 1 — Il setup estrazione dati	16
5.1.2	FASE 2 — Il motore del problema (l'inversa)	17
5.1.3	FASE 3 — La cascata dei calcoli canonici	17
5.1.4	FASE 4 — Scrittura e test di ottimalità	17
5.1.5	FASE 5 — Sensitività (il riciclo)	17

1 Formulario Definitivo: Ricerca Operativa (PL, PLI, Reti)

Questo documento contiene le regole pratiche, i trucchi sui segni, le giustificazioni logiche e numerosi esempi per risolvere gli esercizi di Ottimizzazione a mano.

1.1 Indice

1. Forma standard e dizionario base
2. SBA (Soluzione di Base Ammissibile) e vincoli
3. La forma canonica e l'algebra matriciale
4. Test di ottimalità e regola dei segni
5. Analisi di sensitività e formule matriciali abbreviate
6. Il duale e la regola del "naturale"
7. Scarti complementari (La Regola dell'Incrocio)
8. Branch and Bound (per PL intera)
9. Matrici TUM (Totalmente Unimodulari)
10. TSP e Nearest Neighbor
11. Cammino minimo e Dijkstra
12. Bellman-Ford (cammino minimo con pesi negativi)
13. Linearizzazione e condizioni logiche (PNL $\rightarrow$ PL)
14. Funzione fitness per PSO (metodo delle penalità)
15. Risoluzione step-by-step dell'esercizio di PL

2 PARTE A — Programmazione Lineare (Simplex in forma standard)

2.1 1. Forma standard e dizionario base

Per applicare le matrici, il problema deve avere solo uguaglianze ($=$) e variabili non negative (≥ 0).

Vincolo $\leq$ (Es. risorsa massima): aggiungi uno scarto ($+x_s \geq 0$).

Logica: ne sto usando meno del massimo, per arrivare all'uguale devo aggiungere ciò che avanza.

Vincolo $\geq$ (Es. requisito minimo): sottrai un surplus ($-x_s \geq 0$).

Logica: sto producendo più del minimo, per tornare all'uguaglianza esatta devo togliere l'eccedenza.

ATTENZIONE AI COSTI: le variabili di scarto e surplus non generano né profitto né spesa. Nella F.O., il coefficiente (c) è SEMPRE 0.

Esempio completo.

- (P) $\min z = 3x_1 - 5x_2$
- Vincoli: $2x_1 \leq 4$ e $x_1 - x_2 \geq 1$
- Forma standard: $\min z = 3x_1 - 5x_2 + 0x_3 + 0x_4$
- Vincoli: $2x_1 + x_3 = 4$ e $x_1 - x_2 - x_4 = 1$
- Matrice A :

$$A = \begin{bmatrix} 2 & 0 & 1 & 0 \\ 1 & -1 & 0 & -1 \end{bmatrix}$$

(Le colonne di x_3 e x_4 formano pezzi della matrice identità, a meno del segno.)

Consiglio Pratico d'Esame: quando scrivi la matrice A , allinea bene in colonna le variabili. Se una variabile non compare in un vincolo, metti esplicitamente uno 0 per non sbagliare a estrarre le sottomatrici.

Notarella d'esame. Per i vincoli di uguaglianza ($=$) NON vanno aggiunti né scarto né surplus: il vincolo è già in forma standard. Una variabile “libera” (senza segno), invece, si sdoppia in due variabili non negative $x = x^+ - x^-$ con $x^+, x^- \geq 0$.

2.2 2. SBA (Soluzione di Base Ammissibile) e vincoli

La SBA è geometricamente un vertice del poligono ammissibile. La relazione tra algebra e geometria si basa sul concetto di vincolo attivo/inattivo:

Vincolo ATTIVO: sostituendo il punto, ottieni un'identità (es. $10 = 10$). - Algebra: la variabile di scarto vale ZERO $\Rightarrow$ finisce fuori base (N).

Vincolo INATTIVO: sostituendo il punto, la disuguaglianza è stretta (es. $4 < 10$). - Algebra: la variabile di scarto è MAGGIORE DI ZERO $\Rightarrow$ finisce in base (B).

Esempio completo. Vincolo: $x_1 + 2x_2 \leq 10 \Rightarrow x_1 + 2x_2 + x_3 = 10$.

- Se il vertice è $(4, 3) \Rightarrow 4 + 6 = 10$. Vincolo attivo. Lo scarto è $x_3 = 0$.
- Se il punto è $(1, 1) \Rightarrow 1 + 2 = 3 < 10$. Vincolo inattivo. Lo scarto è $x_3 = 7$.

Consiglio Pratico d'Esame (Soluzioni Degeneri): se hai 3 vincoli, la base DEVE avere 3 variabili. Se trovi che 4 variabili valgono zero (invece delle classiche 2 fuori base), sei su un vertice degenero. Per scegliere la base B , prendi le variabili non nulle e aggiungi quelle nulle fino a formare una matrice A_B con determinante diverso da zero.

Consegna mnemonica. “Numero variabili in base = numero di vincoli (m)”. Nel problema in forma standard hai n variabili e m vincoli: $n - m$ finiscono fuori base ($= 0$), m finiscono in base.

2.3 3. La forma canonica e l'algebra matriciale

Data la base B (indici variabili $\neq 0$) e le fuori base N (variabili $= 0$), i calcoli matriciali servono a riscrivere l'intero sistema rispetto alle variabili in base.

Le Formule Fondamentali.

- Valore della F.O. in base ($\bar{z}$): $\bar{z} = c_B^T A_B^{-1} b$
- Nuova matrice fuori base ($\bar{A}_N$): $\bar{A}_N = A_B^{-1} A_N$
- Vettore dei termini noti / variabili in base ($\bar{b}$): $\bar{b} = A_B^{-1} b$ (per essere ammissibile, tutti i valori in $\bar{b}$ devono essere ≥ 0).
- Costi Ridotti Totali ($\bar{c}^T$): $\bar{c}^T = c^T - c_B^T A_B^{-1} A$.
 - I costi ridotti delle variabili in base si azzerano sempre. Per quelle fuori base, la formula mirata è $\bar{c}_N^T = c_N^T - c_B^T \bar{A}_N$.
 - Il termine intermedio $c_B^T A_B^{-1}$ è il **Vettore Duale**, o **Moltiplicatore**.

Il Sistema in Forma Canonica (Formule Finali Estese). Sostituendo i blocchi calcolati, la rappresentazione esplicita e finale del problema rispetto alla base B isola le variabili in base (x_B) rispetto a quelle fuori base (x_N). Mostrando l'uguaglianza diretta tra la notazione abbreviata e quella matriciale estesa, le formule sono:

$$\min z(x) = \bar{z} + \bar{c}_N^T x_N = (c_B^T A_B^{-1} b) + (c_N^T - c_B^T A_B^{-1} A_N) x_N$$

$$x_B = \bar{b} - \bar{A}_N x_N = (A_B^{-1}b) - (A_B^{-1}A_N)x_N$$

$$x_B, x_N \geq 0$$

(Nota: se il problema è di MAX, la struttura del sistema resta identica, cambierà solo la regola dei segni per valutare l'ottimalità dai costi ridotti.)

Esempio completo (scrittura del sistema). Dai calcoli matriciali:

- $\bar{z} = -85$
- $\bar{b} = \begin{bmatrix} 5 \\ 4 \\ 7 \end{bmatrix}$ con variabili in base $x_B = [x_1, x_2, x_4]^T$
- $\bar{A}_N = \begin{bmatrix} 2/5 & -1/5 \\ -3/5 & 4/5 \\ 7/5 & -11/5 \end{bmatrix}$ con variabili fuori base $x_N = [x_3, x_5]^T$
- $\bar{c}_N^T = [11/5, 7/5]$

La forma canonica finale sarà:

$$\min z(x) = -85 + \frac{11}{5}x_3 + \frac{7}{5}x_5$$

$$x_1 = 5 - \frac{2}{5}x_3 + \frac{1}{5}x_5$$

$$x_2 = 4 + \frac{3}{5}x_3 - \frac{4}{5}x_5$$

$$x_4 = 7 - \frac{7}{5}x_3 + \frac{11}{5}x_5$$

$$x_1, x_2, x_3, x_4, x_5 \geq 0$$

Consiglio Pratico d'Esame (L'inversa 3×3): se devi calcolare l'inversa di una matrice 3×3 a mano, usa il metodo dei cofattori:

$$A_B^{-1} = \frac{1}{\det(A_B)} \cdot (\text{Cofattori})^T$$

Non dimenticare la trasposizione (scambiare righe con colonne) prima di dividere per il determinante, e attento ai segni a scacchiera.

Notarella d'esame. Ricorda che nella forma canonica il “miglioramento” della soluzione corrente dipende solo dalle variabili fuori base: ponendo $x_N = 0$ ottieni immediatamente la SBA corrente ($x_B = \bar{b}$, $z = \bar{z}$).

2.4 4. Test di ottimalità e regola dei segni

Come decido se mi devo fermare? Guardo i **Costi Ridotti** ($\bar{c}_N$).

- **MAX:** ottimo se tutti i $\bar{c}_N \leq 0$. Logica: se forzare l'ingresso di una nuova variabile mi porta un valore negativo (perdita), peggioro. Sono già al massimo.
- **MIN:** ottimo se tutti i $\bar{c}_N \geq 0$. Logica: se inserire una variabile aggiunge un costo positivo, peggioro le spese. Sono già al minimo.

Esempio completo. Problema di MIN. Costi ridotti calcolati: $\bar{c}_N = [3, 0.5]$. Essendo entrambi ≥ 0 , l'ingresso in base di queste variabili farebbe aumentare i costi. La base attuale è ottima.

Notarella d'esame. Un costo ridotto **nullo** per una variabile fuori base segnala una soluzione ottima **alternativa** (stesso valore di z su un altro vertice). Davanti a un $\bar{c}_j = 0$ il Simplex “può entrare” senza peggiorare.

2.5 5. Analisi di sensitività e formule matriciali abbreviate

Regola Mnemotecnica d'Oro:

- Cambiare i **COSTI** (c) $\Rightarrow$ rischio l'**OTTIMALITÀ** (ricalcolo i costi ridotti $\bar{c}_N$).
- Cambiare i **TERMINI NOTI** (b) $\Rightarrow$ rischio l'**AMMISSIBILITÀ** (ricalcolo le x_B).
- Cambiare la **matrice** A (column generation, modifica di una colonna, ...) $\Rightarrow$ può rompere sia ammissibilità che ottimalità, e ricalcolare è molto più delicato. Lo scenario “cambia una colonna di A_B ” reintroduce la necessità di ricalcolare A_B^{-1} .

2.5.1 5.1 Sensitività sui costi (c^{new} o Δc)

Esistono due metodi a seconda di cosa ti fornisce la traccia (un valore esatto o la richiesta di un intervallo). Definiamo $\bar{c}_N$ come il vettore dei costi ridotti attualmente ottimi (già calcolati nella forma canonica).

Metodo 1 (Sostituzione diretta del nuovo valore). Se la traccia fornisce direttamente il nuovo valore esatto di un coefficiente (es. “Cosa succede se il costo c_2 diventa 10?”), puoi evitare i Δ . Inserisci il nuovo valore nel vettore appropriato (c_B^{new} se la variabile è in base, c_N^{new} se è fuori base) e ricalcola i costi ridotti usando la formula originale:

$$\bar{c}_N^{new} = c_N^{new} - (c_B^{new})^T A_B^{-1} A_N$$

Logica: ottenuto il nuovo vettore $\bar{c}_N^{new}$, ti basta guardare i segni. Se rispettano ancora la regola di ottimalità (tutti ≥ 0 per MIN, o ≤ 0 per MAX), la base è rimasta ottima. Altrimenti, l'ottimalità è persa.

Metodo 2 (il trucco veloce con Δc per trovare gli intervalli). Se la traccia chiede “Trovare l'intervallo di stabilità del coefficiente...”, costruisci i vettori Δ posizionando l'incognita al posto giusto e zero altrove. Dobbiamo distinguere due casi.

Variabili NON in base: la variazione altera solo il costo ridotto della variabile specifica. Costruisci il vettore Δc_N (es. se varia c_3 fuori base, avrai $\Delta c_N = [\Delta c_3, 0]$).

- Per MAX: $\Delta c_N \leq -\bar{c}_N$
- Per MIN: $\Delta c_N \geq -\bar{c}_N$

Esempio NON in base (MAX): se $\bar{c}_N = [-3, -2]$ e varia il costo della prima variabile fuori base (c_3). Costruisco $\Delta c_N = [\Delta c_3, 0]$.

$$[\Delta c_3, 0] \leq -[-3, -2] \Rightarrow [\Delta c_3, 0] \leq [3, 2]$$

Ottengo il sistema: $\Delta c_3 \leq 3$ (e $0 \leq 2$, sempre vero). Intervallo per $\Delta c_3 \in]-\infty, 3]$.

Variabili IN base: la variazione provoca un “danno collaterale” su tutto il vettore dei costi ridotti. Costruisci il vettore Δc_B posizionando l’incognita in corrispondenza della posizione che la variabile occupa nella base (es. se varia c_2 che è la seconda variabile in base, avrai $\Delta c_B = [0, \Delta c_2, 0]$).

- Per MAX: $\Delta c_B A_B^{-1} A_N \geq \bar{c}_N$
- Per MIN: $\Delta c_B A_B^{-1} A_N \leq \bar{c}_N$

Esempio IN base (MAX): varia c_2 (seconda in base). Costruisco $\Delta c_B = [0, \Delta c_2, 0]$. Uso la matrice $A_B^{-1} A_N$ già nota e i vecchi costi ridotti $\bar{c}_N = [-3, -2]$.

$$[0, \Delta c_2, 0] \begin{bmatrix} -1/2 & 1/2 \\ 3/4 & -1/4 \\ 9/4 & -7/4 \end{bmatrix} \geq [-3, -2] \implies [\frac{3}{4}\Delta c_2, -\frac{1}{4}\Delta c_2] \geq [-3, -2]$$

Otteni due disequazioni: $\frac{3}{4}\Delta c_2 \geq -3 \implies \Delta c_2 \geq -4$, e $-\frac{1}{4}\Delta c_2 \geq -2 \implies \Delta c_2 \leq 8$. Intervallo: $[-4, 8]$.

2.5.2 5.2 Sensitività sui termini noti (Δb e b^{new})

Per verificare l’ammissibilità, devi imporre che i valori delle variabili in base restino non-negativi ($\bar{b} \geq 0$). Esistono due formulazioni algebriche equivalenti; scegli quella in base ai dati forniti dalla traccia.

Metodo 1 (Sostituzione diretta del vettore completo). Se la traccia ti fornisce direttamente un nuovo vettore dei termini noti b^{new} , calcoli la nuova soluzione in base e imponi che sia ≥ 0 :

$$\bar{b}^{new} = A_B^{-1} b^{new} \geq 0$$

Metodo 2 (il trucco veloce con Δb). Se la traccia richiede di trovare un intervallo per una variazione parametrica Δb , sfrutta la soluzione ottima attuale $A_B^{-1} b$ (ovvero il vettore $\bar{b}$ della forma canonica) che hai già calcolato in precedenza. I nuovi valori in base saranno: $\bar{b}^{new} = \bar{b} + A_B^{-1} \Delta b$. Affinché restino ammissibili (≥ 0), la condizione si riscrive rapidamente come:

$$A_B^{-1} \Delta b \geq -\bar{b}$$

Logica del trucco: affinché la base resti ammissibile, l’impatto della variazione ($A_B^{-1} \Delta b$) non deve abbassare i valori delle variabili oltre il loro valore di base originale invertito di segno ($-\bar{b}$). Questo velocizza enormemente i calcoli perché Δb è quasi sempre un vettore pieno di zeri tranne che in una posizione.

Esempio Metodo 2: Trova l’intervallo per Δb_1 . L’inversa A_B^{-1} la conosci. Il vettore è $\Delta b = [\Delta b_1, 0]^T$. La tua attuale SBA è $\bar{b} = [10, 5]^T$.

$$A_B^{-1} \begin{bmatrix} \Delta b_1 \\ 0 \end{bmatrix} \geq \begin{bmatrix} -10 \\ -5 \end{bmatrix}$$

Moltiplicando la matrice per il vettore con lo zero ottieni subito il sistema di disequazioni finale.

Notarella d’esame. La variazione del termine noto NON cambia i costi ridotti: la base resta o smette di essere **ammissibile**, ma la sua **ottimalità** è intatta finché è ammissibile. Una volta

verificata l'ammissibilità, il nuovo valore della F.O. si legge subito come $\bar{z}^{new} = c_B^T \bar{b}^{new}$ (oppure $\bar{z} + y^T \Delta b$: il guadagno marginale è dato dal moltiplicatore duale).

2.6 6. Il duale e la regola del “naturale”

$MAX \leftrightarrow MIN$ e vettore $c \leftrightarrow$ vettore b (matrice trasposta).

- La natura di un MAX è avere limiti di risorse: $\leq$.
- La natura di un MIN è avere fabbisogni minimi: $\geq$.

Se il vincolo o la variabile del PRIMALE...	Allora nel DUALE genero...
Segue la natura del problema	Qualcosa di Normale (≥ 0 , o vincolo naturale)
Va CONTRO la natura del problema	Qualcosa di Opposto (≤ 0 , o vincolo innaturale)
È un'uguaglianza rigorosa (=)	Una variabile Libera (senza segno)
È una variabile Libera (senza segno)	Un'uguaglianza rigorosa (=) nel vincolo

Esempio completo.

- (P) MAX, $x_1 \geq 0$, $x_2 \leq 0$. Vincolo: $x_1 + x_2 = 10$.
- $\Rightarrow$ (D) MIN. Variabile duale y_1 libera (perché in P c'è un'uguaglianza).
- 1° vincolo duale (legato a $x_1 \geq 0$): $\geq c_1$ (naturale per il MIN).
- 2° vincolo duale (legato a $x_2 \leq 0$): $\leq c_2$ (innaturale per il MIN).

Notarella d'esame. “Easy way” alternativo: conta i vincoli del primale $\rightarrow$ numero di variabili duali; conta le variabili del primale $\rightarrow$ numero di vincoli duali.

2.7 7. Scarti complementari (La Regola dell'Incrocio)

Trova la soluzione del Duale conoscendo il Primale (P) senza usare matrici:

- Se in (P) il vincolo i -esimo è **INATTIVO** ($< o >$) $\Rightarrow$ in (D) $y_i = 0$.
- Se in (P) la variabile $x_j \neq 0 \Rightarrow$ in (D) il vincolo j -esimo è **ATTIVO** (=).

Esempio completo (Il Sistema Magico). Vettore primale $\bar{x} = [4, 0]$. Sappiamo che il vincolo 2 del primale è inattivo $\Rightarrow y_2 = 0$. Poiché $x_1 = 4 \neq 0$, il 1° vincolo del Duale è per forza un'equazione. Sostituisco nel vincolo duale $2y_1 + y_2 \geq c_1$ (poniamo $c_1 = 8$). Diventa: $2y_1 + (0) = 8 \Rightarrow y_1 = 4$. Il vettore duale è $\bar{y} = [4, 0]$. Se rispetta tutti i domini, la $\bar{x}$ primale era ottima!

Notarella d'esame. Incrocio completo: “vincolo attivo $\rightarrow$ variabile duale non nulla ammessa”; “variabile nulla $\rightarrow$ vincolo duale lasco”. La tabella a quattro quadranti à la KKT è simmetrica tra primale e duale scambiati: se conosci $\bar{x}$ ottima ottieni $\bar{y}$, e viceversa.

2.8 8. Branch and Bound (per PL intera)

- **Nodo radice:** risolvi ignorando l'obbligo intero.
- **Branching (ramificazione):** scegli una x con decimali, “taglia” via la parte frazionaria.
- **Fathoming (potatura):** chiudi il ramo (non lo esplori più) se:
 - È inammissibile.
 - Trovi una soluzione 100% intera (diventa il tuo Upper/Lower Bound).

– Il suo potenziale continuo è peggiore di una soluzione intera già trovata.

Esempio completo. Ottimo radice (MAX): $x = 3.6$, $y = 2$, $z = 15.6$. Non è intero. Ramifico su x :

- Ramo 1: aggiungo vincolo $x \leq \lfloor 3.6 \rfloor \implies x \leq 3$.
- Ramo 2: aggiungo vincolo $x \geq \lceil 3.6 \rceil \implies x \geq 4$.

Consiglio Pratico d'Esame: nel Branch and Bound grafico, quando aggiungi un vincolo (es. $x \leq 3$), disegna una retta verticale sul grafico e cancella fisicamente l'area a destra. Il nuovo ottimo sarà quasi sempre all'incrocio tra questa nuova "lama" e uno dei vincoli vecchi!

Notarella d'esame. Il "Bound" è asimmetrico: - Per MAX, il rilassamento continuo è un **Upper Bound** sul valore intero ottimo (dà un limite superiore teorico, perché togliere vincoli $\geq$ non può che migliorare MAX). - Per MIN, il rilassamento continuo è un **Lower Bound**. Per convenzione di letteratura i termini sono invertiti: molti testi chiamano "Lower Bound" una **soluzione intera ammissibile trovata** (è fuorviante), perché per MAX è un **lower bound** sulla quantità che certamente raggiunge l'ottimo. Per evitare confusione, collega sempre "ottimo continuo $\geq$ ottimo intero" per MAX e "ottimo continuo $\leq$ ottimo intero" per MIN.

2.9 9. Matrici TUM (Totalmente Unimodulari)

Una matrice è TUM se il determinante di qualsiasi sua sottomatrice quadrata è 1, -1 oppure 0.

Perché ci interessa? Se la matrice dei vincoli è TUM e il vettore b è intero, **tutti i vertici del poligono sono interi**. Possiamo risolvere il problema come un PL normale e avremo garantita la soluzione intera (saltando il Branch & Bound!).

Come riconosco una matrice TUM all'esame? (Condizione sufficiente.) Una matrice è TUM se rispetta TUTTE e 3 queste regole:

1. I suoi elementi sono solo $+1$, -1 , 0 .
2. In ogni colonna ci sono al massimo due elementi diversi da zero.
3. Le righe possono essere divise in due "squadre" (I e II) tali che:
 - Se in una colonna ci sono due segni uguali (es. due $+1$), devono stare in squadre diverse.
 - Se ci sono due segni diversi (es. un $+1$ e un -1), devono stare nella stessa squadra.

Consiglio Pratico d'Esame: le matrici di incidenza orientate (quelle dei grafi per flusso/cammino minimo) hanno sempre esattamente un $+1$ (freccia che esce) e un -1 (freccia che entra) per ogni arco (colonna). Soddiscano banariamente la regola mettendo tutte le righe nella Squadra I. Sono sempre TUM!

Notarella d'esame. La condizione è **sufficiente** ma NON necessaria: esistono matrici TUM che non soddisfano le tre regole elencate. Se una matrice non le rispetta, NON puoi concludere che non sia TUM — devi verificare i determinanti delle sottomatrici a mano.

3 PARTE B — Reti

3.1 10. Il problema del commesso viaggiatore (TSP) e Nearest Neighbor

Il TSP (Traveling Salesperson Problem) richiede di trovare il percorso più economico che parte da una città, visita tutte le altre città esattamente una volta, e torna al punto di partenza (ciclo Hamiltoniano).

3.1.1 10.1 Spiegazione teorica (Formulazione matematica)

Per spiegare il problema all'esame, devi definire tre cose: le variabili, l'obiettivo e i vincoli.

Dati e Variabili.

- c_{ij} : costo (o distanza) per viaggiare dal nodo i al nodo j .
- $x_{ij} \in \{0, 1\}$: variabile decisionale binaria. Vale 1 se l'arco da i a j fa parte del percorso, vale 0 altrimenti.
- V : insieme dei nodi (le città).
- A : insieme degli archi (i collegamenti disponibili).

Funzione Obiettivo. Si vuole minimizzare il costo totale del viaggio.

$$\min z = \sum_{(i,j) \in A} c_{ij} x_{ij}$$

(Nota matematica: usare l'insieme degli archi A nella singola sommatoria è molto più elegante ed efficiente di una doppia sommatoria generica, perché dice alla matematica di sommare “solo le coppie di nodi tra cui esiste effettivamente una strada”, escludendo a priori collegamenti impossibili o la diagonale.)

I Vincoli Fondamentali.

Vincoli di Assegnazione (entrata e uscita). Devi assicurarti che il commesso entri ed esca da ogni città esattamente una volta.

$$\sum_j x_{ij} = \sum_j x_{ji} = 1 \quad \forall i \in V$$

(Logica: “Per ogni nodo i , le frecce in uscita devono essere uguali alle frecce in entrata, ed entrambe devono valere 1”. È la regola Andata = Ritorno.)

Vincoli di Copertura (eliminazione dei sottocicli tramite tagli). I vincoli di assegnamento da soli non bastano. Il solutore potrebbe creare tanti “mini-giri” disconnessi tra loro (es. un giro chiuso Roma-Napoli-Roma e un giro disconnesso Milano-Torino-Milano), rispettando comunque la regola “entro 1 volta, esco 1 volta”. Per impedirlo, usiamo la formulazione dei tagli (cut-set) appuntata a lezione, valida per archi $i \neq j$:

$$\sum_{(i,j) \in (S, \bar{S})} x_{ij} \geq 1$$

(Logica “delle Isole”: immagina di prendere la mappa e dividerla arbitrariamente in due insiemi: un'isola di città S e l'isola di tutte le altre città $\bar{S}$. Il vincolo impone che ci sia almeno un arco o ponte (≥ 1) che attraversi il confine tra le due isole. Obbligando ogni possibile isoletta a collegarsi col resto del mondo, si rende impossibile la creazione di sotto-cicli isolati. Il percorso sarà per forza un unico grande anello globale.)

3.1.2 10.2 Risoluzione pratica: Euristiche Nearest Neighbor (Vicino più prossimo)

Risolvere il TSP matematicamente è difficilissimo. Per questo si usano delle “euristiche”: regole logiche molto semplici che trovano una soluzione buona (anche se non per forza l'ottima assoluta) in pochissimo tempo. La più famosa è il Nearest Neighbor (l'algoritmo “avidio”).

Algoritmo in 4 passi.

1. **Partenza:** scegli un nodo di partenza a caso (o quello indicato dal prof).
2. **La scelta avida:** guarda i costi verso tutti i nodi che non hai ancora visitato.
3. **Il salto:** vai verso il nodo con il costo più basso, segnati il costo.
4. **Ripeti e chiudi:** ripeti i punti 2 e 3 finché non hai visitato tutti i nodi. Alla fine, somma obbligatoriamente il costo dell'arco che ti riporta dall'ultimo nodo al nodo di partenza.

Esempio pratico da esame (matrice dei costi). Immagina questa matrice delle distanze tra 4 città (A, B, C, D). La diagonale è barrata perché non puoi andare da A ad A.

Nodi	A	B	C	D
A	-	10	15	20
B	10	-	35	25
C	15	35	-	30
D	20	25	30	-

Risoluzione con partenza da A:

1. Sono in A. Guardo la riga di A. I nodi non visitati sono B(10), C(15), D(20). Il minimo è 10. $\Rightarrow$ Vado in B.
2. Sono in B. Guardo la riga di B. Non posso tornare in A. I non visitati sono C(35) e D(25). Il minimo è 25. $\Rightarrow$ Vado in D.
3. Sono in D. Guardo la riga di D. L'unico non visitato rimasto è C. Costo 30. $\Rightarrow$ Vado in C.
4. Sono in C. Li ho visitati tutti. Devo per forza chiudere il ciclo tornando in A. Guardo la riga di C: l'arco $C \rightarrow A$ costa 15.

Soluzione euristica trovata: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$. **Costo totale:** $10 + 25 + 30 + 15 = 80$.

(Nota: siccome è un'euristica "avida", abbiamo fatto scelte ottime all'inizio, ma siamo stati costretti a prendere archi costosi alla fine. È il limite del Nearest Neighbor!)

Notarella d'esame. Il Nearest Neighbor **non è simmetrico** rispetto al nodo di partenza: cambiare città iniziale può cambiare drasticamente il costo finale. Conviene provarlo da più nodi e tenere la soluzione migliore tra quelle trovate ("multi-start NN").

3.2 11. Il problema del cammino minimo e l'algoritmo di Dijkstra

L'obiettivo in questo problema è trovare il percorso meno costoso per spostarsi da un nodo di origine (sorgente s) a un nodo di destinazione finale (d), senza l'obbligo di visitare tutti gli altri nodi.

3.2.1 11.1 Formulazione matematica (teoria)

Per spiegare il problema matematicamente, definiamo gli elementi in gioco. Immagina il cammino minimo come una singola goccia d'acqua (un'unità di flusso) che deve partire dalla sorgente e arrivare a destinazione.

Dati e Variabili.

- c_{ij} : costo (o peso) dell'arco che va dal nodo i al nodo j .
- $x_{ij} \in \{0, 1\}$: variabile binaria. Vale 1 se l'arco fa parte del cammino scelto, 0 altrimenti.
- s : nodo sorgente (partenza).
- d : nodo destinazione.
- V o N : insieme di tutti i nodi del grafo.

- A : insieme di tutti gli archi disponibili.

Funzione Obiettivo. Minimizzare la somma dei costi degli archi scelti.

$$\min z(x) = \sum_{(i,j) \in A} c_{ij} x_{ij}$$

I Vincoli di Bilancio (La Regola dell'Acqua). Il sistema deve garantire che il percorso sia continuo. Si applica a tutti i nodi v del grafo la regola del “Flusso Entrante meno Flusso Uscente”.

$$\sum_{(i,v) \in A} x_{iv} - \sum_{(v,j) \in A} x_{vj} = \begin{cases} -1 & \text{se } v = s \text{ (Sorgente)} \\ 1 & \text{se } v = d \text{ (Destinazione)} \\ 0 & \text{se } v \in V \setminus \{s, d\} \text{ (Nodi intermedi)} \end{cases}$$

Spiegazione della logica dei vincoli.

- **Nodo Sorgente (s):** da qui la goccia deve solo partire, niente entra dall'esterno. Quindi: (entrante 0) – (uscente 1) = –1.
- **Nodo Destinazione (d):** qui la goccia deve arrivare e fermarsi. Quindi: (entrante 1) – (uscente 0) = 1.
- **Nodi intermedi:** tutto ciò che entra deve per forza uscire per continuare il viaggio. Se il nodo viene attraversato: (entrante 1) – (uscente 1) = 0. Se non viene toccato: 0 – 0 = 0.

3.2.2 11.2 Risoluzione pratica: Algoritmo di Dijkstra (pesi positivi)

Attenzione: funziona SOLO con pesi tutti non-negativi. Dijkstra è “avidò”: se chiude un nodo, assume che non si potrà mai trovare una scorciatoia migliore in futuro.

Procedura.

1. **Scegli:** il nodo non visitato con il valore attuale più piccolo, chiudilo con un asterisco *.
2. **Esplora:** guarda i suoi vicini non visitati.
 - Nuovo costo = (costo nodo scelto) + (peso arco).
 - Se nuovo costo < costo attuale, aggiorna: NuovoCosto(NodoScelto).
3. **Backtracking:** dalla destinazione, leggi il nodo tra parentesi e vai a ritroso.

Esempio completo (fase di rilassamento). Sono sul nodo A (scelto in questo step, chiuso a costo definitivo 5). Guardo il suo vicino B (che al momento ha un costo temporaneo di 10(C)). Sulla mappa, l'arco $A \rightarrow B$ pesa 2. Nuovo costo potenziale per B passando da A = 5 + 2 = 7. Poiché 7 < 10, cancello il vecchio valore di B e aggiorno la sua casella scrivendo 7(A).

Notarella d'esame. Il costo “definitivo” scritto su un nodo chiuso non viene MAI più toccato: Dijkstra sblocca i nodi in ordine di distanza crescente, come una “frontiera” che si allarga. Se durante il backtracking devi scegliere la strada giusta basta leggere, a ritroso, l'etichetta del nodo precedente: “(nodoPrecedente)” indica da quale nodo si arriva con il costo minimo.

3.3 12. Algoritmi su reti: Bellman-Ford (Cammino minimo con pesi negativi)

Attenzione: si usa quando la rete ha pesi negativi. A differenza di Dijkstra, Bellman-Ford non “chiude” mai i nodi strada facendo, ma rilassa ripetutamente TUTTI gli archi finché il sistema non si assesta.

Procedura (La Regola del $|V|-1$). Se il grafo ha $|V|$ nodi (es. 5 nodi), l'algoritmo richiederà esattamente $|V| - 1$ iterazioni (es. 4 iterazioni).

1. **Inizializzazione (Iterazione 0):** nodo di partenza = 0, tutti gli altri = $+\infty$.
2. **Iterazioni (da 1 a $|V| - 1$):**
 - Prendi l'elenco di tutti gli archi del grafo.
 - Per ogni arco ($U \rightarrow V$), verifica la condizione di rilassamento:
 - Se $\text{Costo}(U) + \text{Peso}(U, V) < \text{Costo}(V)$, allora $\text{Costo}(V) = \text{Costo}(U) + \text{Peso}(U, V)$ (e segnati che V arriva da U).
 - Ripeti questo intero processo per il numero di iterazioni previsto.
3. **Controllo dei cicli negativi (Iterazione $|V|$):** fai un'ultimissima passata su tutti gli archi. Se anche solo un costo diminuisce ancora, significa che nel grafo c'è un ciclo di peso totale negativo.
 - Logica: se c'è un ciclo negativo, puoi girarci all'infinito abbassando il costo a $-\infty$. Il problema non ammette soluzione.

Esempio completo (un'iterazione). Nodi: S (partenza), A, B. Archi: $S \rightarrow A$ (peso 3), $S \rightarrow B$ (peso 4), $A \rightarrow B$ (peso -2).

- **Iter 0:** S = 0, A = $+\infty$, B = $+\infty$.
- **Iter 1:**
 - Arco $S \rightarrow A$: $0 + 3 < \infty \implies A = 3(S)$.
 - Arco $S \rightarrow B$: $0 + 4 < \infty \implies B = 4(S)$.
 - Arco $A \rightarrow B$: $\text{Costo}(A) + \text{Peso}(A \rightarrow B) = 3 + (-2) = 1$. Poiché $1 < 4$, aggiorniamo $B = 1(A)$.
- **Fine iterazione 1:** i costi sono S = 0, A = 3, B = 1.

Consiglio Pratico d'Esame: quando risolvi Bellman-Ford su carta con la classica tabella iterativa, se durante un'intera iterazione non cambia NESSUN valore, puoi fermarti! Il sistema si è già “assestato” all'ottimo e le iterazioni successive produrranno fotocopie della stessa riga.

Notarella d'esame. L'ordine in cui esami gli archi può cambiare il numero di aggiornamenti in una singola iterazione (dipende dalla topologia), ma $|V| - 1$ iterazioni restano **sempre** sufficienti per propagare l'informazione anche lungo il cammino più lungo. La garanzia “una iterazione di troppo non cambia niente se non ci sono cicli negativi” è il controllo-fai-da-te più semplice.

4 PARTE C — Modelli avanzati e linearizzazioni

4.1 13. Linearizzazione e condizioni logiche (PNL $\rightarrow$ PL)

Spesso i problemi reali presentano condizioni logiche (“se... allora”, “and”, “or”, valori assoluti) o funzioni obiettivo non lineari (come “min-max”). I solutori standard accettano solo funzioni lineari continue. Ecco come “tradurre” le logiche umane in matematica per il computer.

4.1.1 13.1 Vincoli logici tra variabili binarie

Siano $x_\alpha, x_\beta, x_\gamma \in \{0, 1\}$ variabili booleane (1 = vero, 0 = falso).

A) Implicazione logica (IF α THEN β). Vogliamo che se α è vera, allora per forza anche β deve essere vera.

- **Formula:** $x_\alpha \leq x_\beta$.
- Logica: se $x_\alpha = 1$, la formula forza $1 \leq x_\beta \implies x_\beta = 1$. Se $x_\alpha = 0$, allora $0 \leq x_\beta$, lasciando x_β libera di essere 0 o 1.

B) La condizione AND (γ è vera sse ENTRAMBE α e β sono vere).

- **Formule (servono 3 vincoli):**

$$x_\alpha \geq x_\gamma \quad x_\beta \geq x_\gamma \quad x_\gamma \geq x_\alpha + x_\beta - 1$$

- Logica: le prime due regole dicono: “Se γ è accesa (1), allora sia α che β DEVONO essere accese”. La terza regola dice: “Se sia α che β sono accese ($1 + 1 - 1 = 1$), allora γ DEVE accendersi per forza”.

C) La condizione OR (γ è vera se ALMENO UNA tra α e β è vera).

- **Formule (servono 3 vincoli):**

$$x_\alpha \leq x_\gamma \quad x_\beta \leq x_\gamma \quad x_\gamma \leq x_\alpha + x_\beta$$

- Logica: le prime due regole accendono γ non appena o α o β valgono 1. La terza regola costringe γ a spegnersi (0) solo nel caso in cui entrambe α e β siano 0.

D) Vincoli di cardinalità (mutua esclusione tra N variabili).

- “Al massimo una vera” (possono essere anche tutte false): $x_\alpha + x_\beta + x_\gamma \leq 1$.
- “Esattamente una sola vera” (una deve per forza essere vera): $x_\alpha + x_\beta + x_\gamma = 1$.

Notarella d’esame. Attenzione alla direzione delle disuguaglianze di AND/OR: in particolare chi ha $\gamma = \alpha \vee \beta$ di solito sbaglia la **terza** regola — è $x_\gamma \leq x_\alpha + x_\beta$ (che fa spegnere γ se entrambe spente), NON $\geq$. Confondere AND e OR ti distrugge il modello.

4.1.2 13.2 Linearizzazione funzione obiettivo Min-Max (o Max-Min)

Hai una F.O. che ti chiede di minimizzare il valore più grande tra diverse opzioni: $\min \max\{f_1(x), f_2(x), f_3(x)\}$. Non puoi scriverla così nel solutore.

Sostituzione: introduci una nuova variabile continua (chiamiamola K).

- **Nuova F.O.:** $\min K$.
- **Nuovi vincoli:** aggiungi un vincolo per ogni funzione dentro la parentesi:

$$K \geq f_1(x) \quad K \geq f_2(x) \quad K \geq f_3(x)$$

- Logica: poiché stai minimizzando K , il solutore cercherà di abbassare K il più possibile. Ma i vincoli gli impediscono di scendere sotto al valore della funzione più grande tra le tre. In questo modo, “schiacciando” K verso il basso, stai letteralmente minimizzando il massimo.

Notarella d’esame. Per il duale **Max-Min** vale per simmetria: $\max \min\{f_1, f_2, f_3\}$ si linearizza con $\max K$ e vincoli $K \leq f_i(x)$. Per i **vincoli** con Min-Max (non in F.O.), il sistema di disequazioni diventa $f_i(x) \leq K$ per $i = 1, \dots, n$ (niente variabile K da minimizzare: si “espande” in n vincoli). Per il vincolo duale ($\min \leq k$ fissato) scrivi $f_i(x) \leq k$ sempre soltanto $\forall i$.

4.1.3 13.3 Linearizzazione del valore assoluto

Hai un vincolo con un modulo: $|x| \leq m$.

Sostituzione: il valore assoluto si spacca in un sistema di due vincoli lineari:

$$x \leq m \quad -x \leq m \quad (\text{che equivale a } x \geq -m)$$

- Logica: obblighi la variabile a rimanere intrappolata nel range $[-m, +m]$.

Esempio completo (applicazione di TUTTE le regole).*Testo dell'esercizio (PNL):*

$$\min \max\{3y_1 - x_1 + y_3 + y_4, 2y_1 + y_2 + 3x_1 - y_4, y_1 - y_2 + 2y_3 - x_1\}$$

$$s.v. \quad y_1 = 1 \text{ se e solo se } y_2 = 1 \text{ e } y_3 = 1, \quad y_4 = 1 \implies y_3 = 1,$$

$$|x_1| \leq 2, \quad y_i \in \{0, 1\} \text{ con } i = 1, \dots, 4, \quad x_1 \geq 0$$

Risoluzione (modello linearizzato): $\min K$ (sostituzione Min-Max) s.v.**1. Gestione del Min-Max:**

$$K \geq 3y_1 - x_1 + y_3 + y_4 \quad K \geq 2y_1 + y_2 + 3x_1 - y_4 \quad K \geq y_1 - y_2 + 2y_3 - x_1$$

2. Gestione “se e solo se ... e ...” (condizione AND tra y_2 e y_3 collegate a y_1):

$$y_2 \geq y_1 \quad y_3 \geq y_1 \quad y_1 \geq y_2 + y_3 - 1$$

3. Gestione implicazione (IF y_4 THEN y_3):

$$y_4 \leq y_3$$

4. Gestione valore assoluto ($|x_1| \leq 2$):

$$x_1 \leq 2 \quad -x_1 \leq 2 \quad (\text{ridondante perché } x_1 \geq 0, \text{ ma formalmente corretto da scrivere})$$

$$y_1, y_2, y_3, y_4 \in \{0, 1\}, \quad x_1 \geq 0$$

4.2 14. Formulazione funzione fitness per PSO e algoritmi euristici (metodo delle penalità)

Gli algoritmi meta-euristici come il PSO (Particle Swarm Optimization) o gli Algoritmi Genetici sono progettati per esplorare lo spazio alla ricerca di un minimo, ma non sanno gestire i vincoli geometrici. Se un problema ha dei vincoli ($\leq, \geq, =$), dobbiamo trasformarlo in un problema non vincolato creando una singola funzione di fitness $F(x)$ che “penalizzi” le soluzioni inammissibili.

La formula generale (per un problema di MIN):

$$F(x) = f(x) + R \cdot \sum (\text{Penalità})^2$$

(Dove R è un coefficiente costante molto grande, es. 10^4 . Spesso si usa un R_i diverso per ogni vincolo per bilanciare meglio.)

L'algoritmo in 3 step.**PASSO 1 — La funzione obiettivo.** Copia la funzione obiettivo originale $f(x)$ così com'è.

PASSO 2 — Gestione uguaglianze (=). Prendi l'equazione del vincolo e porta tutti i termini a sinistra dell'uguale, in modo da ottenere la forma $h(x) = 0$. La penalità da aggiungere sarà $R \cdot [h(x)]^2$. - Logica: se il vincolo è rispettato, $h(x) = 0$ e la penalità si annulla. Se non lo è, l'elevamento al quadrato renderà l'errore sempre positivo e gravissimo per l'algoritmo (che cerca il minimo).

PASSO 3 — Gestione disuguaglianze ($\leq$ o $\geq$). Porta tutti i termini a sinistra in modo da ottenere SEMPRE la forma $g(x) \leq 0$. (Attenzione: se hai un $\geq$, sposta tutto dall'altra parte dell'uguale o moltiplica tutto per -1 invertendo il segno!) La penalità sarà:

$$R \cdot [\max(0, g(x))]^2$$

- Logica: finché l'espressione è negativa (≤ 0), il vincolo è rispettato; il max sceglierà 0 e non ci sarà alcuna penalità. Se invece il punto “sfiora” diventando positivo (> 0), il max “accende” il termine e scatterà la penalità al quadrato.

Esempio completo (la “ricetta”).

Problema originale:

$$\min f(x) = (x_1 - x_2)^2 + \sin(x_1 \cdot x_2) \quad \text{s.v. } x_1^2 + 3x_2 = 0, \quad 2x_1 - 4x_2 \geq 3$$

Applicazione step-by-step:

1. **F.O.:** resta $(x_1 - x_2)^2 + \sin(x_1 \cdot x_2)$.
2. **Vincolo “=”:** è già nella forma giusta $\Rightarrow$ penalità $R_1 \cdot (x_1^2 + 3x_2)^2$.
3. **Vincolo “ $\geq$ ”:** lo trasformo in ≤ 0 spostando il 3 a sinistra e invertendo i segni $\Rightarrow$ $3 - 2x_1 + 4x_2 \leq 0 \Rightarrow$ penalità $R_2 \cdot [\max(0, 3 - 2x_1 + 4x_2)]^2$.

Funzione fitness finale da passare in pasto al PSO:

$$F(x) = (x_1 - x_2)^2 + \sin(x_1 \cdot x_2) + R_1(x_1^2 + 3x_2)^2 + R_2[\max(0, 3 - 2x_1 + 4x_2)]^2$$

Notarella d'esame. Lo stesso trucco si estende a MAX (uso $F(x) = f(x) - R[\cdot]^2$, perché l'algoritmo cerca il **massimo**) e a **più vincoli** (la sommatoria $\sum_i R_i[\cdot]^2$ li combina tutti). Il quadrato serve a garantire che la penalità sia ≥ 0 in ogni direzione di “violazione”.

5 PARTE D — L'algoritmo infallibile

5.1 15. L'algoritmo infallibile: Risoluzione step-by-step dell'esercizio di PL

Usa questa checklist mentale per affrontare meccanicamente l'esercizio classico di Programmazione Lineare in cui ti viene fornita la base B e ti vengono richiesti la forma canonica, il test di ottimalità e la sensitività. Seguire questo ordine previene la quasi totalità degli errori di distrazione.

5.1.1 FASE 1 — Il setup estrazione dati

1. **Segnati l'obiettivo:** metti un enorme cerchio su MIN o MAX. Tutto dipenderà da questo.
2. **Estrai vettori e matrici:**
 - Estrai il vettore colonna dei termini noti b .
 - Estrai il vettore riga dei costi c^T .

- Estrai l'intera matrice dei vincoli A (se un x manca in un vincolo, ricordati di mettere 0).

3. Dividi le fazioni:

- Prendi gli indici della base B (es. $\{2, 4\}$). Andranno a formare la sottomatrice A_B e il vettore dei costi c_B^T .
- Prendi gli indici rimanenti (le fuori base N , es. $\{1, 3\}$). Formeranno la sottomatrice A_N e il vettore dei costi c_N^T .

5.1.2 FASE 2 — Il motore del problema (l'inversa)

1. **Costruisci fisicamente la matrice A_B** copiando le colonne in ordine.
2. **Calcola l'inversa A_B^{-1} .**
 - Attenzione: questo è il calcolo più pericoloso dell'esame. Se hai una matrice 2×2 , fai $\frac{1}{\det}$ matrice con diagonale principale invertita e segni opposti sull'altra. Ricontrolla bene i segni!

5.1.3 FASE 3 — La cascata dei calcoli canonici

Usa A_B^{-1} appena trovata come un “moltiplicatore magico” per ricalcolare tutto. Calcola tassativamente in questo ordine:

1. **Termini noti in base:** $\bar{b} = A_B^{-1} \cdot b$.
 - *Check rapido:* se ti escono numeri negativi, fermati. Hai sbagliato un segno nell'inversa o la base data non è ammissibile.
2. **Matrice fuori base:** $\bar{A}_N = A_B^{-1} \cdot A_N$.
3. **Vettore moltiplicatore (duale):** $y^T = c_B^T \cdot A_B^{-1}$.
4. **Costi ridotti (fuori base):** $\bar{c}_N^T = c_N^T - y^T \cdot A_N$.
5. **Funzione obiettivo in base:** $\bar{z} = c_B^T \cdot \bar{b}$.

5.1.4 FASE 4 — Scrittura e test di ottimalità

1. **Scrivi il sistema.** Ricopia la forma canonica “riempiendola” con i numeri calcolati in Fase 3:

$$\text{Obiettivo: } z(x) = \bar{z} + \bar{c}_N^T x_N$$

$$\text{Vincoli: } x_B = \bar{b} - \bar{A}_N x_N$$

2. **Il test.** Guarda i numeri dentro il vettore $\bar{c}_N^T$.
 - Il problema era di **MIN**? Per essere ottimo devono essere tutti ≥ 0 .
 - Il problema era di **MAX**? Per essere ottimo devono essere tutti ≤ 0 .

Scrivi esplicitamente sul foglio: “Poiché i valori di $\bar{c}_N$ rispettano la condizione ≥ 0 (per MIN) oppure ≤ 0 (per MAX), la SBA è ottima”.

5.1.5 FASE 5 — Sensitività (il riciclo)

Non ricalcolare mai nulla da capo. Usa le frazioni trovate in Fase 3! Costruisci i vettori Δ mettendo le incognite nella giusta posizione e gli 0 altrove.

- Se varia un **costo IN base** (c_2): usa $\Delta c_B \cdot \bar{A}_N \geq \bar{c}_N^T$ (per MAX) o $\leq \bar{c}_N^T$ (per MIN). Risolvi il sistema di disequazioni.
- Se varia un **costo FUORI base** (c_1): usa $\Delta c_N \leq -\bar{c}_N^T$ (per MAX) o $\geq -\bar{c}_N^T$ (per MIN). Risoluzione solitamente immediata.

- Se varia una **risorsa** (b_1): usa $A_B^{-1} \cdot \Delta b \geq -\bar{b}$. Risolvi il sistema di disequazioni.

Notarella d'esame (chiusura). Tre trappole classiche: 1. Confondere MIN/MAX nella regola dei segni (è il motivo n. 1 di esami persi). 2. Dimenticarsi di **azzerare** i costi ridotti delle variabili in base quando si ricalcolano per la sensitività su Δc_B . 3. Usare la c_B senza la trasposta nella formula $\bar{z} = c_B^T A_B^{-1} b$ (compatibilità con la F.O. e con il moltiplicatore duale $y^T = c_B^T A_B^{-1}$).